Table 3. Performance of DPOR algorithms on two benchmarks.

	Traces Explored			Time		
Benchmark	classic	source	optimal	classic	source	optimal
filesystem(14)	4	2	2	0.54s	0.36s	0.35s
filesystem(16)	64	8	8	8.13s	1.82s	1.78s
filesystem(18)	1024	32	32	2m11s	8.52s	8.86s
filesystem(19)	4096	64	64	8m33s	18.62s	19.57s
indexer(12)	78	8	8	0.74s	0.11s	0.10s
indexer(15)	341832	4096	4096	56m20s	50.24s	52.35s

Table 3 shows the number of traces that the three algorithms explore as well as the time it takes to explore them. It is clear that our algorithms, which in these benchmarks explore the same (optimal) number of interleavings, beat 'classic' DPOR with sleep sets, by a margin that becomes wider as the number of threads increases. As a sanity check, Kähkönen et al. [2012] report that their unfolding-based method is also able to explore only 8 paths for indexer(12), while their prototype implementation of DPOR extended with sleep sets and support for commutativity of reads and writes explores between 51 and 138 paths (with 85 as median value). The numbers we report (78 for 'classic' DPOR and 8 for our algorithms) are very similar.

Performance on two synthetic benchmarks. Next, we compare the algorithms on two synthetic benchmarks that expose differences between them. The first is the readers program of Fig. 13. The results, for 2, 8 and 13 readers are shown in Table 4. For 'classic' DPOR the number of explored traces is $O(3^{\mathsf{N}})$ here, while source- and optimal-DPOR only explore 2^{N} traces. Both numbers are exponential in N but, as can be seen in the table, for e.g., $\mathsf{N} = 13$ both source-DPOR and optimal-DPOR finish in about one and a half minute, while the DPOR algorithm with the sleep set extension [Flanagan and Godefroid 2005a] explores two orders of magnitude more (mostly sleep set blocked) traces and needs almost one and a half hours to complete.

Table 4. Performance of DPOR algorithms on more benchmarks.

	Traces Explored			Time		
Benchmark	classic	source	optimal	classic	source	optimal
readers(2)	5	4	4	0.02s	0.02s	0.02s
readers(8)	3281	256	256	13.98s	1.31s	1.29s
readers(13)	797162	8192	8192	86m 7s	1m26s	1m26s
lastzero(5)	241	79	64	1.08s	0.38s	0.32s
lastzero(10)	53198	7204	3328	4m47s	45.21s	27.61s
lastzero(15)	9378091	302587	147456	1539m11s	55m 4s	30m13s

The second benchmark is the lastzero(N) program presented earlier in Fig. 11. As can be seen in Table 4, source-DPOR explores about twice as many traces as optimal-DPOR and, naturally, even if it uses a cheaper test, takes almost twice as much time to complete.